

“Capture the Flag 4: The Student List”

Goal: Build a scrollable student directory using Dart Lists and `ListView.builder`, replacing manually duplicated widgets with dynamically generated UI.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before building your student list.

Create:

`feature/student-list`

Screenshot:

- Terminal showing your current branch

```
student@PC-06 MINGW64 ~/IT314AB_KONG/IT314AB_KONG4/IT314AB_KONG (main)
$ git checkout -b feature/student-list
Switched to a new branch 'feature/student-list'
```

🚩 Flag 1 — Build Your Student Data

Objective: Store multiple students inside a Dart `List`.

Research what a Dart `List` is and why it is useful when handling multiple pieces of similar data.

Instead of creating one student at a time with separate variables, create a collection that stores at least five students.

Each student should contain information such as:

- Image
- Name
- Course
- Year Level
- Age
- Hobby

Your data should exist separately from your UI widgets.

Screenshot:

Code:

- Your student list
- At least five student entries

```
8 List<Map<String, dynamic>> students = [
9   {
10    "image": "assets/cutie.jpg",
11    "name": "Julia Yashika Kong",
12    "course": "BSIT",
13    "yearLevel": 3,
14    "age": 20,
15    "hobby": "Eating",
16   },
17   {
18    "image": "assets/meow.png",
19    "name": "Emji Silento",
20    "course": "BSIT",
21    "yearLevel": 3,
22    "age": 22,
23    "hobby": "Drinking Alcohol",
24   },
25   {
26    "image": "assets/ha.jpg",
27    "name": "Leachim Dela Cerna",
28    "course": "BSIT",
29    "yearLevel": 3,
30    "age": 21,
31    "hobby": "Dancing",
32   },
33   {
34    "image": "assets/pretty.jpg",
35    "name": "Karryle Racaza",
36    "course": "BSIT",
37    "yearLevel": 3,
38    "age": 21,
39    "hobby": "Dancing",
40   },
41   {
42    "image": "assets/sexy.jpg",
43    "name": "Mav Sumalangitka",
44    "course": "BSIT",
45    "yearLevel": 3,
46    "age": 21,
47    "hobby": "Dancing & Singing",
48   },
49 ];
```

🚩 Flag 2 — The Copy-Paste Trap

You already know how to create a Student Card.

You created five student cards manually.

You duplicated the same card design until all five students appear.

Stop.

Imagine having:

- 50 students.
- 500 students.
- 5,000 students.

Would copying the same widget over and over still be practical?

This flag is intentionally making you feel the problem before solving it.

Screenshot:

- Your five manually created student cards (do not change your past code yet)

```
81 Card(  
82   margin: const EdgeInsets.all(10),  
83   child: Padding(  
84     padding: const EdgeInsets.all(10),  
85  
86     child: Row(  
87       crossAxisAlignment: CrossAxisAlignment.start,  
88  
89       children: [  
90         Image.asset(  
91           students[0]["image"],  
92           width: 80,  
93           height: 80,  
94           fit: BoxFit.cover,  
95  
96           errorBuilder: (context, error, stackTrace) {  
97             return Container(  
98               width: 80,  
99               height: 80,  
100              color: Colors.grey[300],  
101              child: const Icon(  
102                Icons.person,  
103                size: 50,  
104              ), // Icon  
105            ); // Container  
106          },  
107        ), // Image.asset
```

```

150 Card(
151   margin: const EdgeInsets.all(10),
152   child: Padding(
153     padding: const EdgeInsets.all(10),
154
155     child: Row(
156       crossAxisAlignment: CrossAxisAlignment.start,
157
158       children: [
159         Image.asset(
160           students[1]["image"],
161           width: 80,
162           height: 80,
163           fit: BoxFit.cover,
164
165           errorBuilder: (context, error, stackTrace) {
166             return Container(
167               width: 80,
168               height: 80,
169               color: Colors.grey[300],
170               child: const Icon(
171                 Icons.person,
172                 size: 50,
173               ), // Icon
174             ); // Container
175           },
176         ), // Image.asset
177
178         const SizedBox(width: 15),

```

```

219 Card(
220   margin: const EdgeInsets.all(10),
221   child: Padding(
222     padding: const EdgeInsets.all(10),
223
224     child: Row(
225       crossAxisAlignment: CrossAxisAlignment.start,
226
227       children: [
228         Image.asset(
229           students[2]["image"],
230           width: 80,
231           height: 80,
232           fit: BoxFit.cover,
233
234           errorBuilder: (context, error, stackTrace) {
235             return Container(
236               width: 80,
237               height: 80,
238               color: Colors.grey[300],
239               child: const Icon(
240                 Icons.person,
241                 size: 50,
242               ), // Icon
243             ); // Container
244           },
245         ), // Image.asset
246

```

```

287 Card(
288   margin: const EdgeInsets.all(10),
289   child: Padding(
290     padding: const EdgeInsets.all(10),
291
292     child: Row(
293       crossAxisAlignment: CrossAxisAlignment.start,
294
295       children: [
296         Image.asset(
297           students[3]["image"],
298           width: 80,
299           height: 80,
300           fit: BoxFit.cover,
301
302           errorBuilder: (context, error, stackTrace) {
303             return Container(
304               width: 80,
305               height: 80,
306               color: Colors.grey[300],
307               child: const Icon(
308                 Icons.person,
309                 size: 50,
310               ), // Icon
311             ); // Container
312           },
313         ), // Image.asset
314
315         const SizedBox(width: 15),

```

```

355 Card(
356   margin: const EdgeInsets.all(10),
357   child: Padding(
358     padding: const EdgeInsets.all(10),
359
360     child: Row(
361       crossAxisAlignment: CrossAxisAlignment.start,
362
363       children: [
364         Image.asset(
365           students[4]["image"],
366           width: 80,
367           height: 80,
368           fit: BoxFit.cover,
369
370           errorBuilder: (context, error, stackTrace) {
371             return Container(
372               width: 80,
373               height: 80,
374               color: Colors.grey[300],
375               child: const Icon(
376                 Icons.person,
377                 size: 50,
378               ), // Icon
379             ); // Container
380           },
381         ), // Image.asset
382
383         const SizedBox(width: 15),
384

```

🚩 Flag 3 — Discover `ListView`

Objective: Make your student directory scrollable using `ListView`.

Research Flutter's `ListView`.

Replace your previous layout with a scrolling layout so users can browse through all of your student cards naturally.

Your application should now allow continuous vertical scrolling.

Test

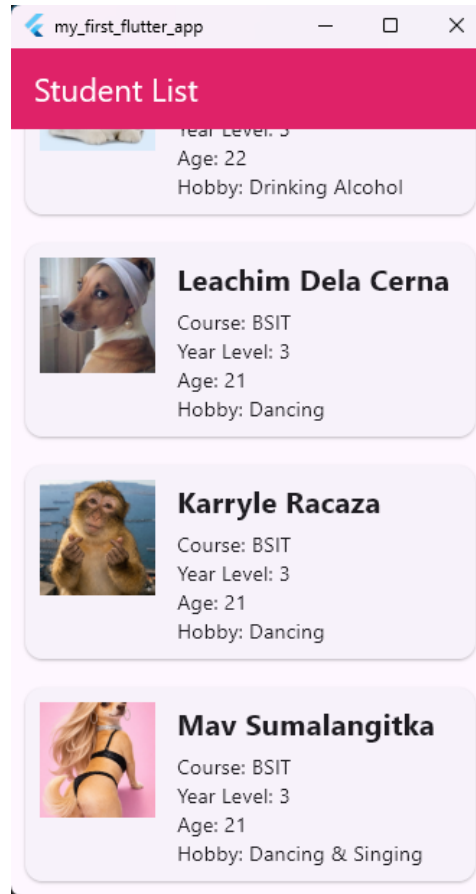
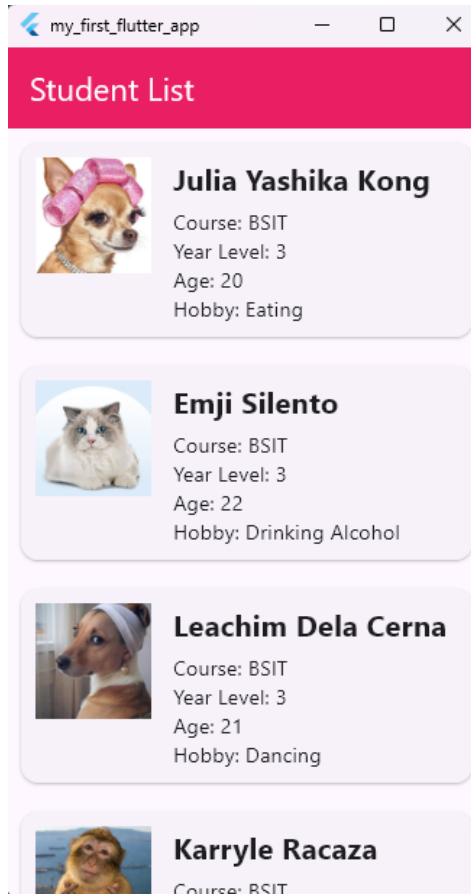
The user should be able to:

- Scroll from the first student
- Continue scrolling
- Reach the last student without resizing the screen

Screenshot:

- `ListView` in your code
- Top of the application
- Bottom portion showing later students

```
75     body: ListView(  
76       children: [  
77         Card(  
78           margin: const EdgeInsets.all(10),  
79           child: Padding(  
80             padding: const EdgeInsets.all(10),  
81             child: Row(  
82               crossAxisAlignment: CrossAxisAlignment.start,  
83               children: [  
84                 Image.asset(  
85                   students[0]["image"],  
86                   width: 80,  
87                   height: 80,  
88                   fit: BoxFit.cover,  
89                   errorBuilder: (context, error, stackTrace) {  
90                     return Container(  
91                       width: 80,  
92                       height: 80,  
93                       color: Colors.pink,  
94                       child: const Icon(  
95                         Icons.person,  
96                         size: 50,  
97                       ), // Icon  
98                     ); // Container  
99                   },  
100                 ), // Image.asset  
101               ],  
102             ),  
103           ),  
104         ],  
105       ),
```



🚩 Flag 4 — Unlock `ListView.builder`

Objective: Generate student cards automatically using `ListView.builder`.

Research `ListView.builder`.

Instead of writing five separate Student Cards, generate them automatically from your student list.

Your UI should now create one card for each student in your list.

Test

Add one new student to your list.

Without creating another card manually, your application should display the new student automatically.

Screenshot:

- `ListView.builder` in your code
- Student list
- Newly generated student appearing in the app

```

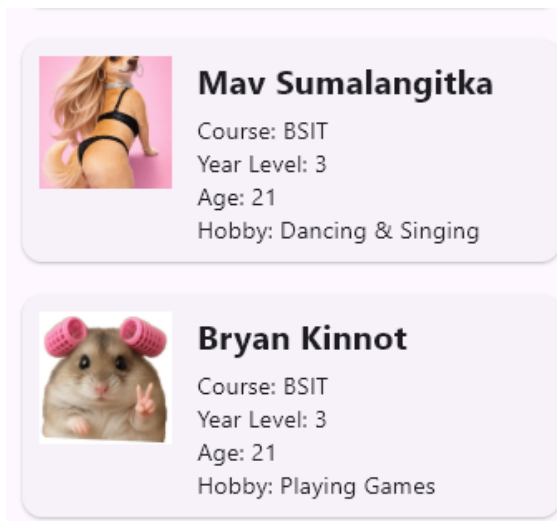
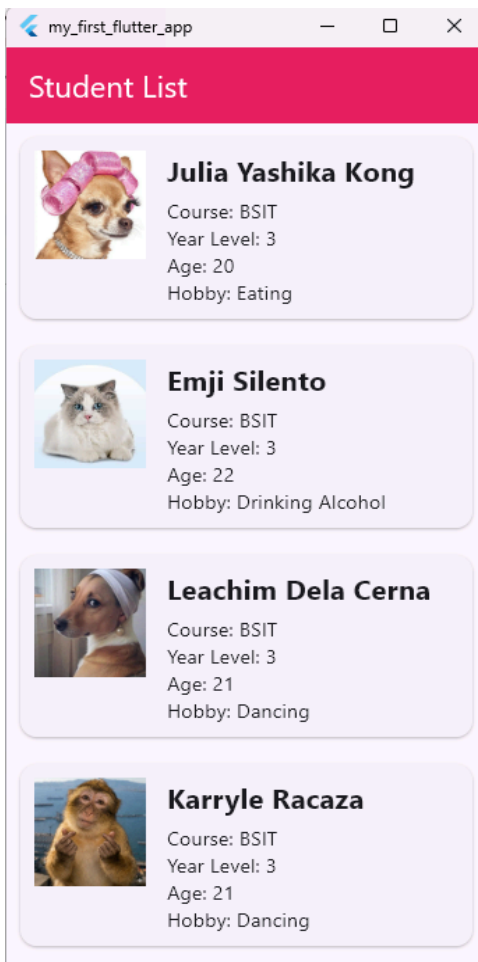
84   body: ListView.builder(
85     itemCount: students.length,
86
87     itemBuilder: (context, index) {
88       return Card(
89         margin: const EdgeInsets.all(10),
90
91         child: Padding(
92           padding: const EdgeInsets.all(10),
93
94           child: Row(
95             crossAxisAlignment: CrossAxisAlignment.start,
96
97             children: [
98               Image.asset(
99                 students[index]["image"],
100                width: 80,
101                height: 80,
102                fit: BoxFit.cover,

```

```

48 //NEW STUDENT ADDED
49 {
50   "image": "assets/mice.jpg",
51   "name": "Bryan Kinnot",
52   "course": "BSIT",
53   "yearLevel": 3,
54   "age": 21,
55   "hobby": "Playing Games",
56 },
57 ];

```



🚩 Flag 5 — Every Student Deserves an ID

Objective: Expand your student records with additional information.

Real student directories contain more than just names.

Add at least three new fields to every student.

Examples:

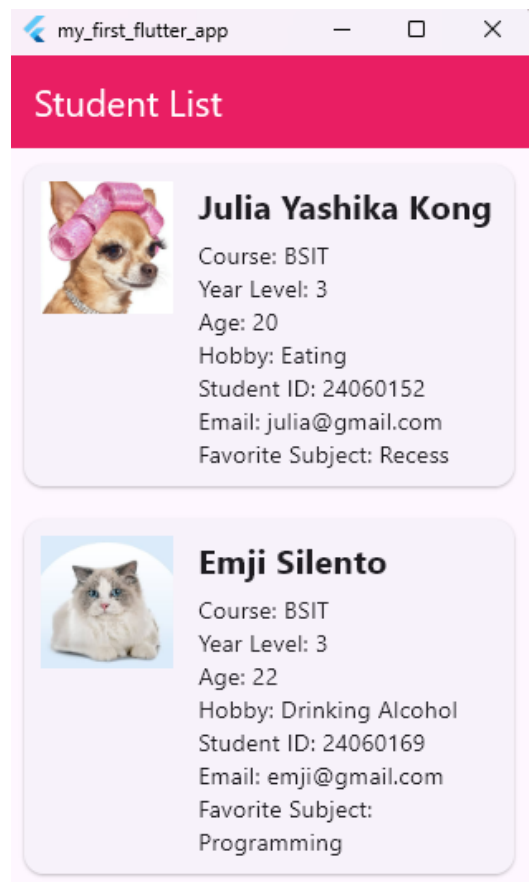
- Student ID
- Email
- Favorite Subject

Update your Student Card so the new information appears consistently across every generated card.

Screenshot:

- Updated student data
- Student Card displaying the additional fields

```
7 List<Map<String, dynamic>> students = [  
8   {  
9     "image": "assets/cutie.jpg",  
10    "name": "Julia Yashika Kong",  
11    "course": "BSIT",  
12    "yearLevel": 3,  
13    "age": 20,  
14    "hobby": "Eating",  
15    "studentID": "24060152",  
16    "email": "julia@gmail.com",  
17    "favoriteSubject": "Recess",  
18  },  
19  {  
20    "image": "assets/meow.png",  
21    "name": "Emji Silento",  
22    "course": "BSIT",  
23    "yearLevel": 3,  
24    "age": 22,  
25    "hobby": "Drinking Alcohol",  
26    "studentID": "24060169",  
27    "email": "emji@gmail.com",  
28    "favoriteSubject": "Programming",  
29  },  
]
```



🚩 Flag 6 — Empty Classroom

What happens when there are no students?

Objective: Handle an empty `List` gracefully.

Imagine your app just connected to a database.

The database returns:

```
[]
```

No students.

What should the user see?

Instead of displaying a broken or confusing screen, research how Flutter can display a different widget when a condition is met.

Create an empty state that clearly communicates that there are currently no students available.

Examples:

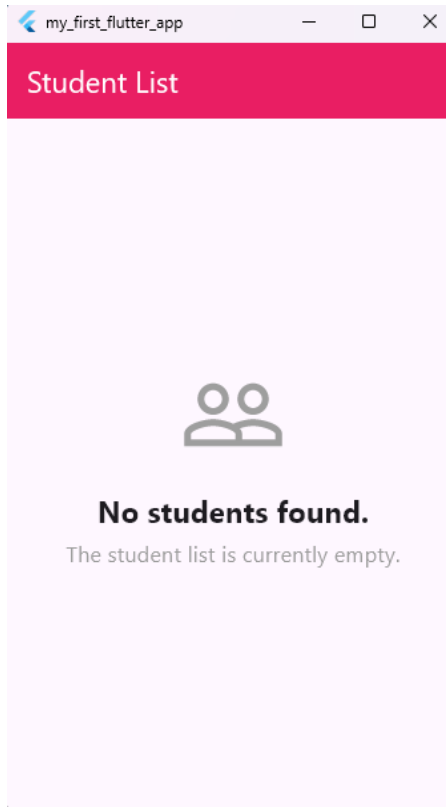
- "No students found."
- "Student list is empty."

Test

- Display your student list normally.
- Then temporarily make your list empty.
- Your application should display the empty state instead of student cards.

Screenshot:

- Empty student list
- Empty-state UI
- Application still functioning correctly



```
35 |         body: students.isEmpty
36 |           ? const Center(
37 |             child: Column(
38 |               mainAxisAlignment: MainAxisAlignment.center,
39 |               children: [
40 |                 Icon(
41 |                   Icons.people_outline,
42 |                   size: 80,
43 |                   color: Colors.grey,
44 |                 ), // Icon
45 |
46 |                 SizedBox(height: 15),
47 |
48 |                 Text(
49 |                   "No students found.",
50 |                   style: TextStyle(
51 |                     fontSize: 22,
52 |                     fontWeight: FontWeight.bold,
53 |                   ), // TextStyle
54 |                 ), // Text
```

```
List<Map<String, dynamic>> students = [];

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: "Student List",
      home: const StudentPage(),
    ); // MaterialApp
  }
}
```

🚩 Flag 7 — Sort the Roll Call

Objective: Change the order of your student list.

Class lists are rarely displayed randomly.

Research how Dart can rearrange items inside a `List`.

Sort your students alphabetically by name.

Test

Before sorting:

- Maria
- John
- Aaron
- Sophia

After sorting:

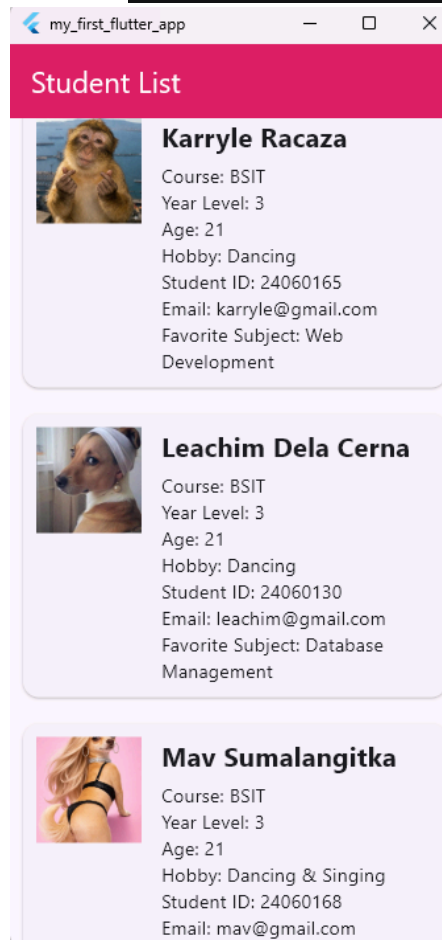
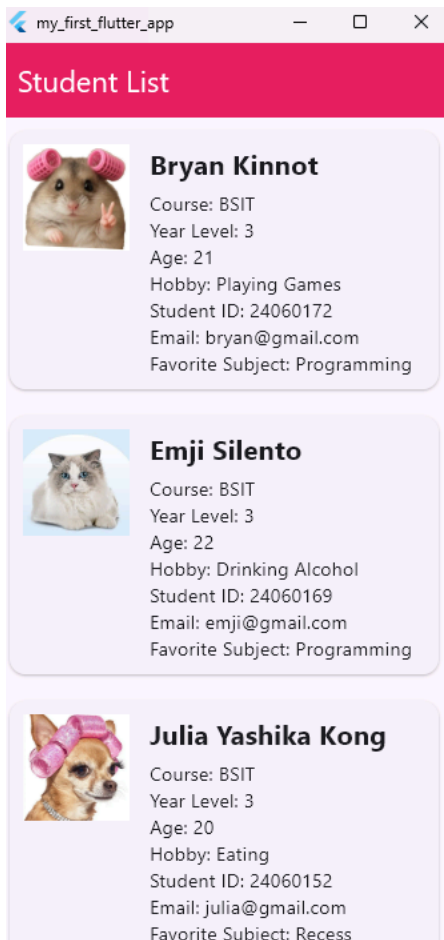
- Aaron
- John
- Maria
- Sophia

Your UI should immediately reflect the new order.

Screenshot:

- Student list before or after sorting
- Application displaying the sorted order

```
76 void sortStudents() {  
77     students.sort(  
78         (a, b) => a["name"].compareTo(b["name"]),  
79     );  
80 }
```



🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Student List CTF.

Use an appropriate commit message following proper Git standards.

Branch:

feature/student-list

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

```
PS C:\Users\student\IT314AB_KONG\IT314AB_KONG4\IT314AB_KONG\my_first_flutter_app> git add .
warning: in the working copy of 'my_first_flutter_app/linux/flutter/generated_plugin_registrant.cc', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'my_first_flutter_app/linux/flutter/generated_plugin_registrant.h', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'my_first_flutter_app/linux/flutter/generated_plugins.cmake', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'my_first_flutter_app/macos/Flutter/GeneratedPluginRegistrant.swift', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'my_first_flutter_app/windows/flutter/generated_plugin_registrant.cc', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'my_first_flutter_app/windows/flutter/generated_plugin_registrant.h', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'my_first_flutter_app/windows/flutter/generated_plugins.cmake', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\student\IT314AB_KONG\IT314AB_KONG4\IT314AB_KONG\my_first_flutter_app> git commit -m "feat:feature and student list flutter project"
[feature/student-list eecd2de] feat:feature and student list flutter project
9 files changed, 222 insertions(+), 188 deletions(-)
 create mode 100644 my_first_flutter_app/assets/cutie.jpg
 create mode 100644 my_first_flutter_app/assets/ha.jpg
 create mode 100644 my_first_flutter_app/assets/mice.jpg
 create mode 100644 my_first_flutter_app/assets/pretty.png
 create mode 100644 my_first_flutter_app/assets/sexy.jpg
PS C:\Users\student\IT314AB_KONG\IT314AB_KONG4\IT314AB_KONG\my_first_flutter_app> git push origin feature/student-list
info: please complete authentication in your browser...
Enumerating objects: 22, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (14/14), 737.08 KiB | 184.27 MiB/s, done.
Total 14 (delta 5), reused 5 (delta 1), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 5 local objects.
remote:
remote: Create a pull request for 'feature/student-list' on GitHub by visiting:
remote:   https://github.com/yashipikapa/IT314AB_KONG/pull/new/feature/student-list
remote:
To https://github.com/yashipikapa/IT314AB_KONG.git
 * [new branch]   feature/student-list -> feature/student-list
```